# Введение в разработку программного обеспечения

Перед написанием и выполнением нашей первой программы, мы должны понять, как вообще выполняется разработка программного обеспечения на языке C++.

**Схема разработки ПО** (сокр. от «*Программное Обеспечение*»):

## Оглавление:


- **Шаг №1: Определите проблему, которую хотели бы решить**
- **Шаг №2: Определитесь, как вы собираетесь решить эту проблему**
- **Шаг №3: Напишите программу**
- **Шаг №4: Компиляция**
- **Шаг №5: Линкинг (связывание объектных файлов)**
- **Шаг №6: Тестирование и отладка**


## Шаг №1: Определите проблему, которую хотели бы решить

Это шаг «**Что?**». Здесь вы должны понять, что же вы хотите, чтобы ваша программа делала. Этот шаг может быть, как самым простым, так и самым сложным. Всё, что вам нужно — это четко сформулировать идею. Только после этого вы сможете приступать к следующему шагу.

Вот несколько примеров выполнения шага №1:

- «Я хочу написать программу, которая вычисляла бы среднее арифметическое чисел, которые я введу».
- «Я хочу написать программу, в которой будет 2D-лабиринт, по которому сможет передвигаться пользователь».
- «Я хочу написать программу, которая будет анализировать цены акций на бирже и давать предсказания по поводу скачков этих цен вверх или вниз».

## Шаг №2: Определитесь, как вы собираетесь решить эту проблему

Здесь мы уже отвечаем на вопрос «**Как?**». Каким образом можно решить проблему, обозначенную на шаге №1? Этим шагом довольно часто пренебрегают при разработке программного обеспечения. Суть в том, что способов решения задачи может быть много, только часть из них — хорошие решения, а часть — плохие. Вы должны научиться отделять первые от вторых. Очень часто можно наблюдать ситуацию, когда у программиста возникает идея и он сразу же садится программировать. Как вы уже могли догадаться, такой сценарий далеко не всегда приводит к эффективным результатам.

Как правило, **хорошие решения имеют следующие характеристики**:

- простота;
- хорошая документация (с инструкциями и комментариями);
- модульный принцип: любая часть программы может быть повторно использована или изменена позже, не затрагивая другие части кода;
- надежность: соответствующая обработка ошибок и экстренных ситуаций.

Когда вы садитесь и начинаете сразу программировать, вы думаете: «*Я хочу сделать это, вот это и еще вот это!*». Таким образом вы принимаете решения, которые позволят вам поскорее выполнить задание. Однако это может привести к тому, что вы получите программу, которую позже будет трудно изменить/модифицировать, добавить что-то новое или вам попросту придется разбираться с большим количеством **багов**.

Согласно **закону Парето**, программист тратит примерно 20% времени на написание программы и 80% времени на отладку (исправление ошибок) или поддержку (добавление новых функциональных возможностей) кода. Следовательно, лучше потратить дополнительное время на обдумывание лучшего способа решения проблемы перед процессом написания кода, нежели потом тратить оставшиеся 80% времени на поиск и исправление ошибок.

## Шаг №3: Напишите программу

Для того, чтобы написать программу, необходимы две вещи:

- знание выбранного вами языка программирования (этому мы вас научим);
- редактор кода.

Программу можно написать, используя любой редактор, даже тот же *Блокнот* в Windows или текстовый редактор *Vi* в Unix. Тем не менее, в настоятельно рекомендую использовать редактор, предназначенный для программирования. Не беспокойтесь, если у вас его еще нет. На следующем уроке мы рассмотрим процесс установки такого приложения.

Редактор типичного программиста, как правило, имеет следующие особенности, которые облегчают программирование:

- **Нумерация строк**. Это функция чрезвычайно полезна при отладке программ, когда компилятор выдаёт нам сообщения об ошибках. Типичная ошибка компиляции состоит из наименования ошибки и номера строки, где эта ошибка произошла (например, «*ошибка переопределения переменной x, строка 90*»). Без нумерации строк искать ту самую 90-ю строку кода было бы несколько затруднительно, не так *ли*?
- **Подсветка синтаксиса**. Подсветка синтаксиса изменяет цвет разных частей программы и кода, что улучшает восприятие как целой программы, так и её отдельных частей.
- **Специальный шрифт**. В обычных шрифтах очень часто возникает путаница между определенными символами, когда непонятно, какой символ перед вами. Например: цифра `0` или буква `O`, цифра `1` или буква `l` (верхний регистр `L`), или может буква `I` (нижний регистр `i`). Вот для этого и нужен специальный шрифт, в котором будет легко различить эти символы, предотвращая случайное использование одного символа вместо другого.

Программы на языке C++ следует называть ***name.cpp***, где *name* заменяется именем вашей программы, а расширение *.cpp* сообщает компилятору (и вам тоже), что это исходный файл кода, который содержит инструкции на языке программирования C++. Следует обратить внимание, что некоторые программисты используют расширение *.cc* вместо *.cpp*, но я рекомендую использовать именно *.cpp*.

Также стоит отметить, что много программ, написанных на языке C++, могут состоять из нескольких файлов *.cpp*. Хотя большинство программ, которые вы будете создавать на протяжении этих уроков, не будут слишком большими, в дальнейшем вы научитесь писать программы, которые будут включать десятки, если не сотни отдельных *.cpp*-файлов.

## Шаг №4: Компиляция

Для того, чтобы скомпилировать программу нам нужен **компилятор**. **Работа компилятора состоит из двух частей**:

- Проверка программы на соответствие правилам языка C++ (проверка синтаксиса). Если она будет неудачной, то компилятор выдаст сообщения об ошибках, которые нужно будет исправить.
- Конвертация каждого исходного файла с кодом в **объектный файл** (или ***«объектный модуль»***) на машинном языке. Объектные файлы, как правило, имеют названия *name.o* или *name.obj*, где *name* должно быть такое же как и имя вашего исходного *.cpp*-файла. Если ваша программа состоит из трех *.cpp*-файлов, то компилятор сгенерирует 3 объектных файла.

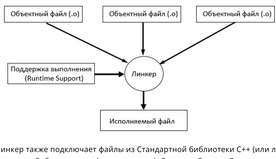
Стоит отметить, что такие операционные системы как **Linux** и macOS имеют уже встроенный компилятор C++, который называется **g++**. Для компиляции файлов из командной строки с помощью g++ вам нужно будет написать следующее:

```
g++ -c file1.cpp file2.cpp file3.cpp
```

Таким образом мы создадим file1.o, file2.o и file3.o. `-c` означает «только скомпилировать», т.е. просто создать *.o* (объектные) файлы. Кроме g++, существует множество компиляторов для различных операционных систем: Linux, Windows, macOS и других.

## Шаг №5: Линкинг (связывание объектных файлов)

**Линкинг** — это процесс связывания всех объектных файлов, генерируемых компилятором, в единую исполняемую программу, которую вы затем сможете запустить/выполнить. Это делается с помощью программы, которая называется **линкер** (или ***«компоновщик»***).




Кроме объектных файлов, линкер также подключает файлы из Стандартной библиотеки C++ (или любой другой библиотеки, которую вы используете, например, библиотеки графики или звука). Сам по себе язык C++ довольно маленький и простой. Тем не менее, к нему подключается большая библиотека дополнительных функций, которые могут использовать ваши программы, и эти функции находятся в Стандартной библиотеке C++. Например, если вы хотите вывести что-либо на экран, то у вас в коде должна быть специальная команда, которая сообщит компилятору, что вы хотите использовать функцию вывода информации на экран из Стандартной библиотеки C++.

После того, как компоновщик закончит линкинг всех объектных файлов (при условии, что не будет ошибок), вы получите исполняемый файл. Опять же, в целях наглядности, чтобы связать *.o*-файлы, которые мы создали выше в Linux или macOS, мы можем снова использовать g++:

```
g++ -o prog file1.o file2.o file3.o
```

Команда `-o` сообщает g++, что мы хотим получить исполняемый файл с именем `prog` из следующих файлов: file1.o, file2.o и file3.o. При желании, компиляцию и линкинг можно объединить в один шаг:

```
g++ -o prog file1.cpp file2.cpp file3.cpp
```

Результатом будет исполняемый файл с именем `prog`.

## Шаг №6: Тестирование и отладка

Здесь начинается самое веселое! Вы уже можете запустить исполняемый файл и посмотреть, работает ли всё так, как надо. Если нет, то пришло время отладки. Более детально об отладке мы поговорим на соответствующих уроках.

Обратите внимание, для выполнения шагов №3-№6 вам потребуется специальное программное обеспечение. Хотя вы можете использовать отдельные программы на каждом из этих шагов, один пакет программного обеспечения (сокр. ***«IDE»*** от англ. «*Integrated Development Environments*») объединяет в себе все эти программы. Обычно с IDE вы получаете редактор кода с нумерацией строк и подсветкой синтаксиса, а также компилятор и линкер. А когда вам нужно будет провести отладку программы, вы сможете использовать встроенный отладчик. Кроме того, IDE объединяет и ряд других полезных возможностей: комплексная помощь, дополнение кода, в некоторых случаях еще и система контроля версий.